

Politechnika Poznańska Wydział Automatyki, Robotyki i Elektrotechniki Automatyka i Robotyka		
Dz>AiR>S5>SM	Systemy Mikroprocesorowe (SM)	2025/26 (s.zim.)
Skład osobowy: Bartosz Terlecki Kacper Ratuszny	System regulacji temperatury z interfejsem graficznym	

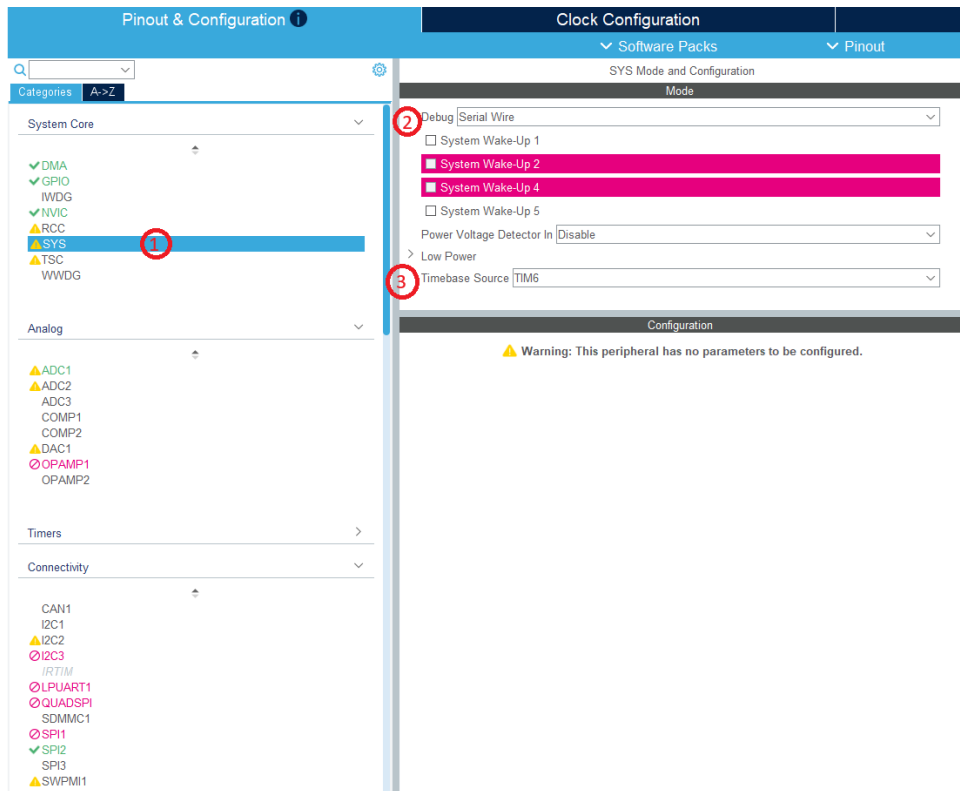
Spis treści

1	Wprowadzenie	2
2	Przebieg projektu	2
2.1	Schemat elektryczny	2
2.2	Konfiguracja w STM32CubeMX	2
2.2.1	Konfiguracja FreeRTOS	2
2.2.2	Czujnik temperatury LM35DZ	4
2.2.3	Wyświetlacz ST7735S	4
2.2.4	Konfiguracja elementu grzewczego	4
2.2.5	Konfiguracja elementu chłodzącego	5
2.3	Etapy tworzenia projektu - Regulacja	5
2.3.1	Wykreślenie odpowiedzi skokowej obiektu	5
2.3.2	Identyfikacja modeli grzałki i wentylatora	7
2.3.3	Projektowanie regulatorów	8
2.4	Wprowadzone funkcjonalności i opis działania kodu	8
2.4.1	Sterowanie wyświetlaczem ST7735S	8
2.4.2	Aplikacja desktopowa i interfejs komunikacyjny	9
2.4.3	Pętla główna programu i reszta funkcjonalności	10
2.4.4	Reszta funkcjonalności	11
3	Wnioski	11
3.1	Zrealizowane założenia projektowe	11
3.2	Wnioski końcowe	11
4	Bibliografia	12

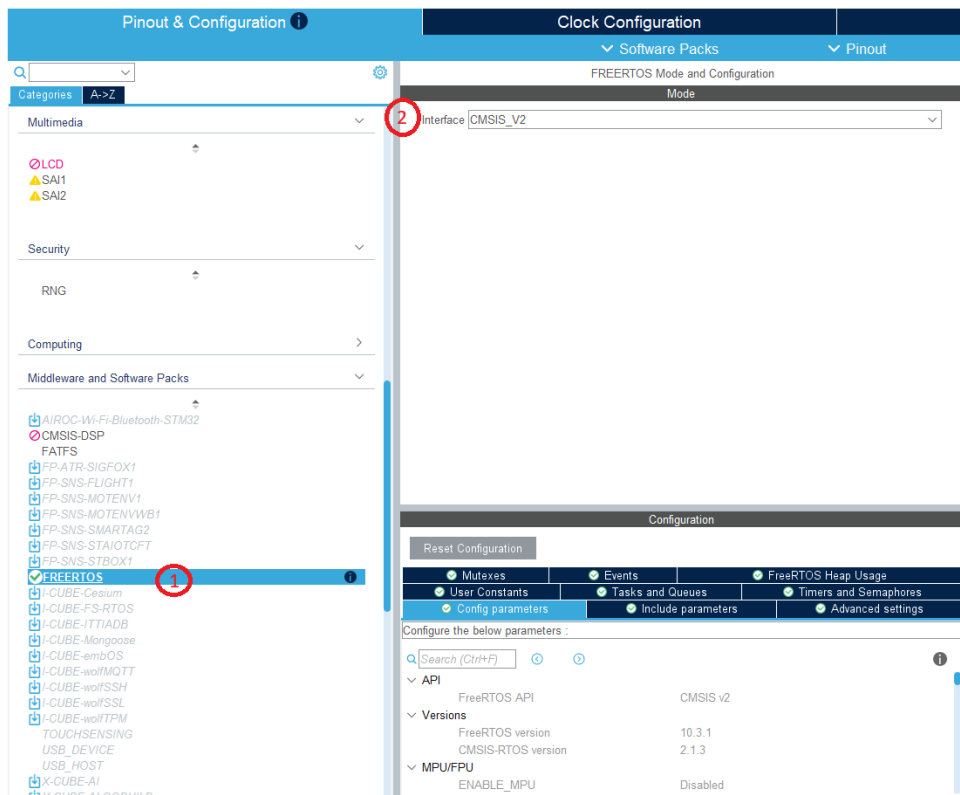
Spis rysunków

1	Ideowy schemat elektryczny projektu	2
2	Nieprzefiltrowana odpowiedź na wymuszenie skokowe	6
3	Porównanie różnych rzędów filtrów SMA	6
4	Porównanie odpowiedzi modelu z danymi eksperymentalnymi dla grzałki i wentylatora	7
5	Aplikacja desktopowa	9

Spis tabel



Następnie w zakładce Middleware and Software Packs należy aktywować FreeRTOS oraz wybrać interfejs CMSIS-RTOS v2, który jest nowszą i obecnie zalecaną wersją. Interfejs CMSIS-RTOS v1 jest niewspierany i niezalecany do stosowania w nowych projektach.



Po włączeniu FreeRTOS konieczne jest również sprawdzenie jego podstawowej konfiguracji. Należy upewnić się, że przydzielony rozmiar heapu jest wystarczający dla tworzonych zadań, a w projekcie istnieje przynajmniej jedno zadanie (np. zadanie domyślne generowane przez STM32CubeMX). Warto także zweryfikować rozmiar stosu przypisanego do zadania, ponieważ zbyt mała wartość może prowadzić do błędów wykonania, takich jak HardFault.

2.2.2 Czujnik temperatury LM35DZ

Układ wykorzystuje dwa czujniki LM35DZ w obudowie TO-92, umieszczone pod rezystorem ceramicznym. Czujniki generują napięcie proporcjonalne do temperatury, 10 mV na każdy stopień Celsjusza. Pierwszy czujnik podłączono do pinu PC0 (ADC1_IN1), a drugi do PC1 (ADC1_IN2), przy czym oba kanały pracują w trybie *single-ended*. ADC1 działa w trybie ciągłej konwersji z DMA w trybie cyklicznym, co pozwala automatycznie przysyłać wyniki pomiarów do pamięci. Oversampling w stosunku 16 z przesunięciem 4 bitów poprawia dokładność, a czas próbkowania 640 cykli zapewnia stabilne odczyty. Dzięki temu system może nieprzerwanie monitorować temperaturę, a otrzymane napięcia są przeliczane na stopnie Celsjusza¹.

2.2.3 Wyświetlacz ST7735S

Wyświetlacz został podłączony do mikrokontrolera przez interfejs SPI2 w trybie *Transmit Only Master*, co oznacza, że mikrokontroler wysyła dane do wyświetlacza bez odbierania odpowiedzi. Linia MOSI podłączona jest do pinu PC3, a linia zegara SCK do pinu PB10. Sterowanie wyświetlaczem realizowane jest poprzez dodatkowe linie: PB11 pełni funkcję DC (Data/Command), PB12 służy jako CS (Chip Select), a PB2 odpowiada za sygnał resetu. Takie rozwiązanie pozwala na szybkie i niezawodne przysyłanie danych graficznych i tekstowych na wyświetlacz, zapewniając płynną wizualizację pomiarów temperatury oraz innych parametrów systemu².

2.2.4 Konfiguracja elementu grzewczego

Element grzewczy sterowany jest sygnałem PWM generowanym przez timer TIM1, kanał 1 (TIM1_CH1), który został skonfigurowany jako PWM Generation CH1. Sygnał PWM pozwala na regulację mocy dostarczanej do grzałki poprzez modulację wypełnienia. Preskaler timera ustawiono na 79, a wartość okresu (ARR) na 999 przy częstotliwości zegara timera $f_{\text{timer}} = 80 \text{ MHz}$. Częstotliwość PWM oblicza się ze wzoru:

$$f_{\text{PWM}} = \frac{f_{\text{timer}}}{(\text{Prescaler} + 1) \cdot (\text{ARR} + 1)}$$

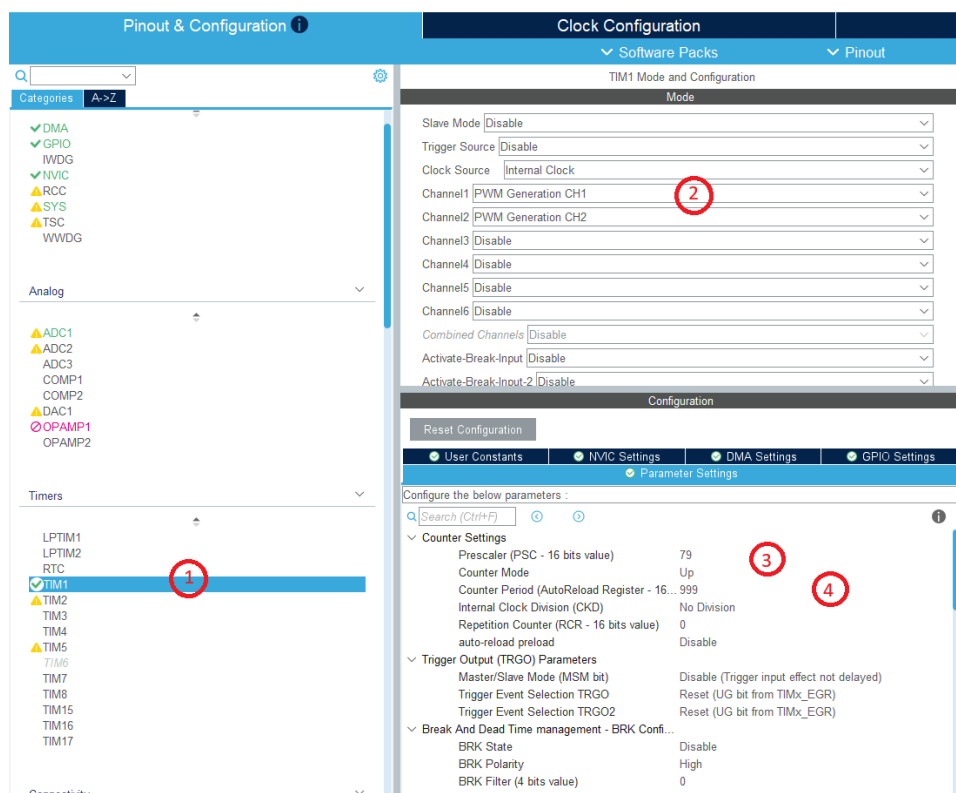
Podstawiając wartości:

$$f_{\text{PWM}} = \frac{80\,000\,000}{(79 + 1) \cdot (999 + 1)} = \frac{80\,000\,000}{80\,000} = 1000 \text{ Hz}$$

Otrzymany sygnał PWM o częstotliwości 1 kHz zapewnia stabilne i precyzyjne sterowanie temperaturą grzałki.

¹Opracowano zgodnie z treścią bloga FORBOT: <https://forbot.pl/blog/kurs-stm3214-przetworniki-analogowo-cyfrowe-adc-id46587>, wątek „Odczyt z wielu kanałów ADC”

²Po podłączeniu i bardziej obszerne wyjaśnienie można znaleźć w źródle: <https://forbot.pl/blog/kurs-stm3214-kolorowy-wyswietlacz-tft-spi-id48620>



2.2.5 Konfiguracja elementu chłodzącego

Element chłodzący sterowany jest sygnałem PWM z TIM1, kanał 2 (TIM1_CH2). Wypełnienie sygnału reguluje prędkość wentylatora, a częstotliwość jest taka sama jak dla elementu grzewczego.

2.3 Etapy tworzenia projektu - Regulacja

2.3.1 Wykreślenie odpowiedzi skokowej obiektu

W pierwszej kolejności podłączyłem i skonfigurowałem wejścia ADC zgodnie z konfiguracją w CubeMX oraz schematem elektrycznym. Aby pomiar był bardziej dokładny w całej długości rezystora, przygotowałem funkcję uśredniającą wyniki z dwóch czujników.

Napięcie z czujnika obliczane jest ze wzoru:

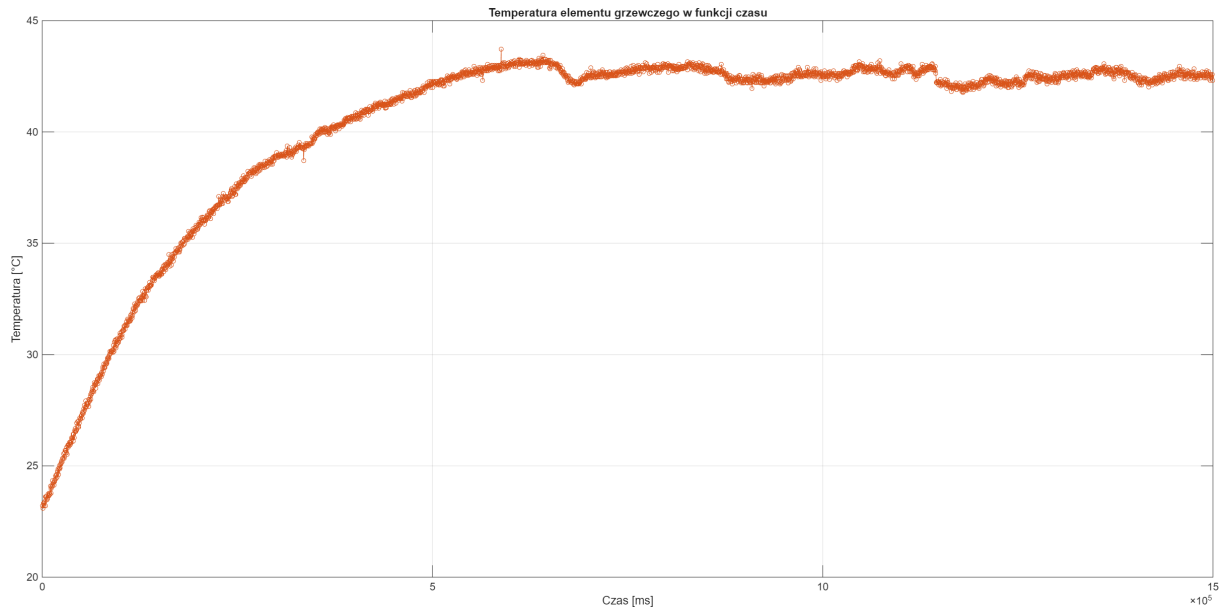
$$V_{ADC} = \frac{ADC_{raw} \cdot V_{ref}}{4095}$$

gdzie ADC_{raw} to wartość odczytana z przetwornika ADC, a V_{ref} to napięcie odniesienia.

Temperatura w stopniach Celsjusza jest następnie obliczana na podstawie napięcia wyjściowego czujnika LM35:

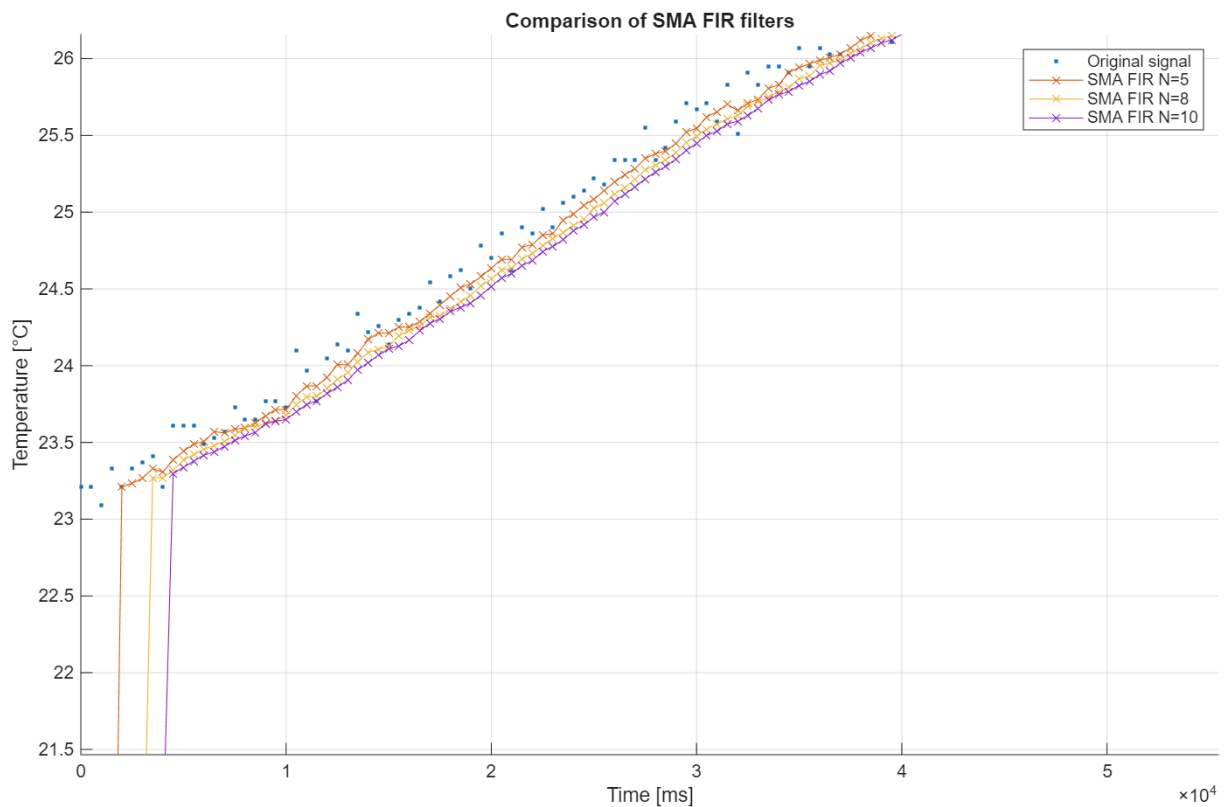
$$T[^{\circ}C] = V_{ADC} \cdot 100$$

Po podłączeniu rezystora i ustawieniu maksymalnego wypełnienia PWM napisałem skrypt w MATLABie do odczytu danych i wykreślenia odpowiedzi skokowej. Wyniki przedstawiono na rysunku 2:



Rysunek 2: Nieprzefiltrowana odpowiedź na wymuszenie skokowe

Następnie zaprojektowałem filtr uśredniający SMA, którego parametry automatycznie wygenerowały wartości filtra i kod źródłowy w katalogach `Components/inc` oraz `Components/Src`. Rząd filtra można było dobrać w MATLABie, co pozwoliło porównać wpływ różnych parametrów filtra na wygładzanie odpowiedzi (rysunek 3).



Rysunek 3: Porównanie różnych rzędów filtrów SMA

2.3.2 Identyfikacja modeli grzałki i wentylatora

Na podstawie przefiltrowanych odpowiedzi skokowych w MATLABie wyznaczono parametry dynamiczne obiektów. Modele zostały dopasowane do danych eksperymentalnych przy użyciu funkcji `tfest` z toolboxa System Identification.

Otrzymane parametry przedstawiono poniżej:

- **Grzałka:**

$$K_{\text{heater}} = 0.000244 \text{ } ^\circ\text{C/PWM}, \quad \tau_{\text{heater}} = 0.006 \text{ s}, \quad T_{d,\text{heater}} = 0 \text{ s}$$

- **Wentylator:**

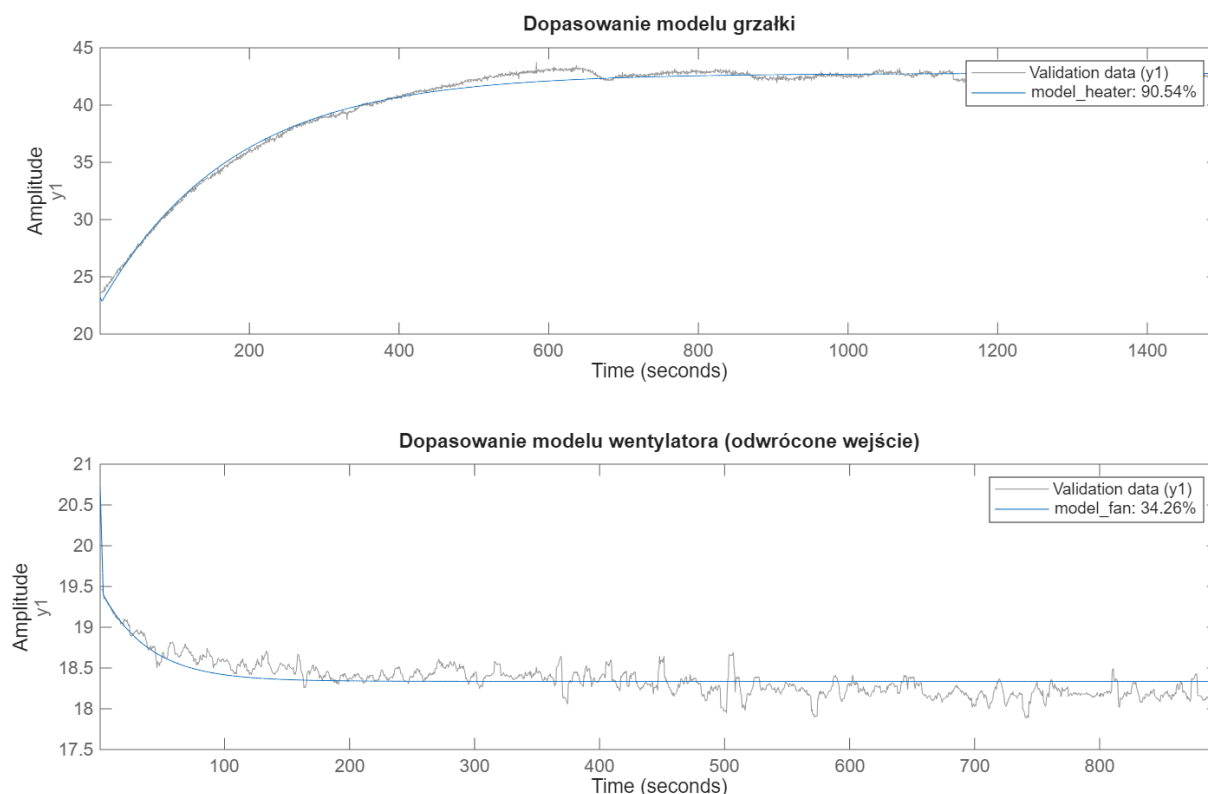
$$K_{\text{fan}} = -0.000487 \text{ } ^\circ\text{C/PWM}, \quad \tau_{\text{fan}} = 0.027 \text{ s}, \quad T_{d,\text{fan}} = 0 \text{ s}$$

Modele w postaci transmitancji ciągłych opisują zależność temperatury od sterowania PWM:

$$G_{\text{heater}}(s) = \frac{K_{\text{heater}}}{\tau_{\text{heater}}s + 1}, \quad G_{\text{fan}}(s) = \frac{K_{\text{fan}}}{\tau_{\text{fan}}s + 1}$$

Modele zostały zapisane w pliku `identified_models.mat` i wykorzystane w dalszej części pracy do symulacji i projektowania regulatora PID.

Dla wizualnej weryfikacji dopasowania modeli do danych eksperymentalnych przygotowano wykresy porównawcze (rysunek 4):



Rysunek 4: Porównanie odpowiedzi modelu z danymi eksperymentalnymi dla grzałki i wentylatora

Jak widać na rysunku 4, model grzałki dobrze odwzorowuje dynamikę układów w obszarze pracy bliskim punktowi pracy, co pozwala na dalsze projektowanie regulatora PID.

Natomiast dla wentylatora dopasowanie jest słabe, więc tutaj od razu przechodzę do metody prób i błędów.

2.3.3 Projektowanie regulatorów

Po zidentyfikowaniu modeli dynamicznych grzałki oraz wentylatora zaprojektowano regulatory PID w środowisku MATLAB, wykorzystując transmitancje obiektów. Celem regulacji było uzyskanie możliwie szybkiej odpowiedzi układu przy jednoczesnym ograniczeniu przeregulowania oraz oscylacji.

Regulator PID grzałki:

$$K_p = 33.4694,$$

$$K_i = 0.3455,$$

$$K_d = 0$$

Regulator PID wentylatora:

$$K_p = 78.0725,$$

$$K_i = 3.7582,$$

$$K_d = 0$$

Uzyskane parametry regulatorów nie zapewniły jednak zadowalającej jakości regulacji w warunkach rzeczywistych (wentylator od początku słabo dopasowany). W związku z tym dokonano ich dalszego strojenia metodą prób i błędów - dla grzałki jedynie korekta K_p , w celu poprawy stabilności układu oraz jakości odpowiedzi dynamicznej.

2.4 Wprowadzone funkcjonalności i opis działania kodu

2.4.1 Sterowanie wyświetlaczem ST7735S

Moduł ST7735S.c obsługuje wyświetlacz TFT ST7735S przez interfejs SPI i został podzielony na funkcje niskopoziomowe, rysowanie grafiki oraz wizualizację temperatury i PWM.

Funkcje niskiego poziomu Zapewniają bezpośrednią kontrolę wyświetlacza:

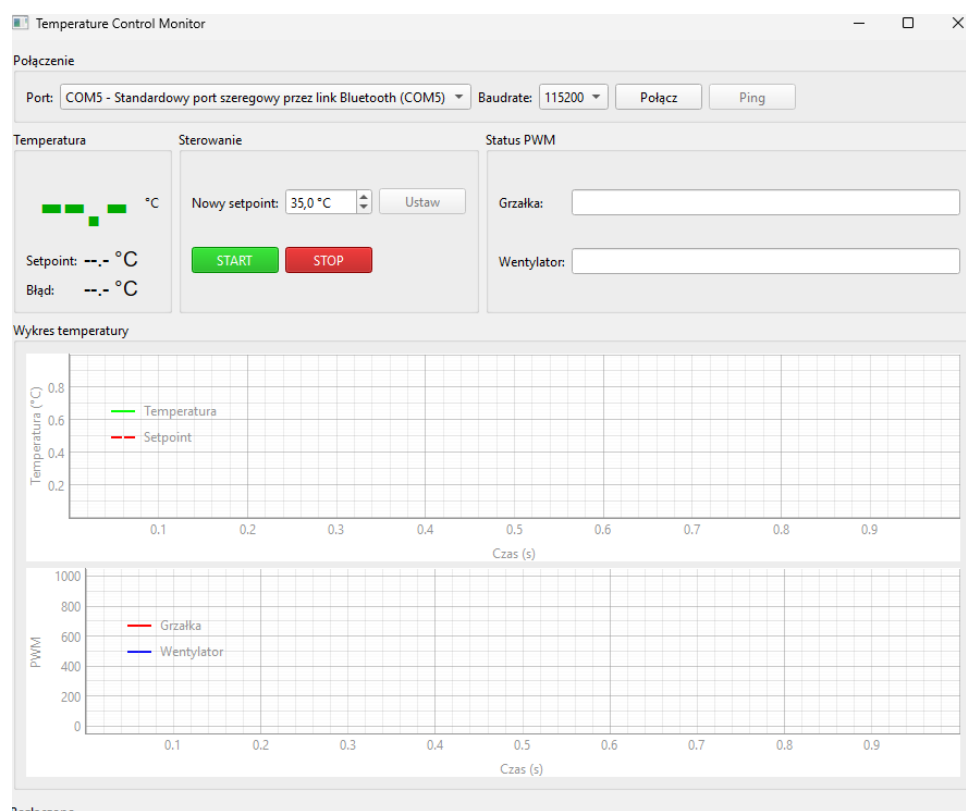
- aktywacja/dezaktywacja (ST7735_Select/Unselect),
- wybór trybu danych/komendy (ST7735_DC_Command/Data),
- reset sprzętowy (ST7735_Reset),
- przesyłanie komend i danych (ST7735_WriteCommand/Data),
- ustawienie okna rysowania (ST7735_SetAddressWindow).

Inicjalizacja ST7735_Init wykonuje reset, wysyła sekwencję komend inicjalizacyjnych (tryb kolorów RGB565, zasilanie, gamma, orientacja) i włącza wyświetlacz z czarnym tłem.

Rysowanie i wizualizacja Moduł udostępnia funkcje do rysowania pikseli, linii, prostokątów, znaków i liczb (`DrawPixel`, `DrawRect`, `DrawString`, `DrawInt/Float`, `FillScreen`). Kolory definiowane są w formacie RGB565, a `ST7735_TempColor` mapuje temperaturę na skalę od niebieskiego do czerwonego. Funkcje `DrawProgressBar` i `ShowTemperature` wyświetlają aktualną temperaturę, setpoint, błąd regulacji oraz wartości PWM grzałki i wentylatora.

Podsumowanie Separacja komunikacji SPI od funkcji graficznych zwiększa wydajność i ułatwia rozbudowę modułu. Obsługa kolorów i czcionki 5x7 pozwala na dynamiczną wizualizację stanu urządzenia.

2.4.2 Aplikacja desktopowa i interfejs komunikacyjny



Rysunek 5: Aplikacja desktopowa

Aplikacja desktopowa została zaimplementowana w Pythonie z wykorzystaniem biblioteki `PyQt6` i modułu `pyqtgraph` do wizualizacji danych w czasie rzeczywistym. Jej głównym zadaniem jest monitorowanie temperatury oraz sterowanie urządzeniem poprzez interfejs UART, zgodnie z protokołem opisanym w module `uart_protocol.c`.

Wątek komunikacji szeregowej Do obsługi komunikacji szeregowej wykorzystano dedykowany wątek (`SerialThread`), który działa w tle i zapewnia:

- Połączenie i rozłączenie z wybranym portem szeregowym, obsługę parametrów takich jak `baudrate`.
- Odbiór danych w formacie `$DATA,temp,setpoint,heater_pwm,fan_pwm,timestamp` oraz sygnałów sterujących (`$ACK`, `$NAK`, `$PONG`).
- Weryfikację integralności danych poprzez `CRC8`.

- Emitowanie sygnałów Qt (`pyqtSignal`) do aktualizacji interfejsu użytkownika w czasie rzeczywistym.

Interfejs użytkownika Okno główne aplikacji (`TemperatureMonitor`) oferuje:

- Panel połączenia z portem szeregowym, umożliwiający wybór portu i `baudrate`, przycisk `Ping` oraz status połączenia.
- Panel monitorowania temperatury, wyświetlający aktualną temperaturę, wartość zadanej temperatury (`setpoint`) oraz błąd regulacji z kolorową sygnalizacją odchylenia.
- Panel sterowania, pozwalający ustawić nowy `setpoint`, uruchomić lub zatrzymać urządzenie oraz wysłać komendy `START` i `STOP`.
- Panel statusu PWM grzałki i wentylatora, pokazujący aktualne wartości sterowania w formie pasków postępu.
- Dynamiczne wykresy temperatury i PWM w czasie rzeczywistym z wykorzystaniem `pyqtgraph`, uwzględniające ostatnie 5 minut pomiarów.

Integracja z protokołem UART Aplikacja implementuje te same zasady protokołu UART co moduł `uart_protocol.c` w mikrokontrolerze:

- Obliczanie i weryfikacja CRC8 dla każdej wysyłanej i odbieranej wiadomości.
- Tworzenie i wysyłanie komend `$PING`, `$SETP`, `$START`, `$STOP` itp.
- Odbiór danych telemetrycznych i sygnałów potwierdzających (`ACK`, `NAK`), które są automatycznie przetwarzane i wyświetlane w interfejsie.

Dzięki takiemu podziałowi na wątek komunikacyjny i interfejs użytkownika aplikacja zapewnia płynne odświeżanie danych, niezależne od szybkości transmisji UART, oraz umożliwia pełną kontrolę nad urządzeniem w czasie rzeczywistym.

2.4.3 Pętla główna programu i reszta funkcjonalności

Główna pętla programu jest realizowana w środowisku FreeRTOS poprzez zadania i timery. Do kluczowych funkcji należą:

- **StartDefaultTask** – pętla główna, utrzymuje wątek aktywnym.
- **start_usart_com** – obsługa komunikacji UART, odbiór i parsowanie komend, aktualizacja `setpoint`, sterowanie start/stop oraz wysyłanie danych pomiarowych.
- **LcdTask** – aktualizacja wyświetlacza ST7735S z wartością temperatury, `setpoint` oraz PWM grzałki i wentylatora co 500 ms.
- **temp_callback** – timer odczytujący temperaturę, aktualizujący regulator PID i ustawiający PWM grzałki oraz wentylatora.

Timer, kolejka i przerwanie UART Program wykorzystuje w FreeRTOS następujące mechanizmy:

- **Timer `temp_meas`** – okresowy timer wywołujący funkcję `temp_callback` co 1000 ms. Funkcja ta odczytuje średnią temperaturę z czujników, aktualizuje regulator PID, ustawia PWM grzałki i wentylatora oraz zapisuje dane do zmiennych globalnych dla innych zadań.
- **Kolejka `uartRxQueue`** – bufor FIFO przechowujący bajty odbierane z UART. Zadanie `start_usart_com` odczytuje kolejkę, parsuje komendy i wykonuje odpowiednie akcje (np. zmiana setpoint, START/STOP urządzenia, wysyłanie danych pomiarowych).
- **Przerwanie UART** – funkcja `HAL_UART_RxCpltCallback` jest wywoływana po odebraniu pojedynczego bajtu z UART. Bajt trafia do kolejki `uartRxQueue`, a następnie następuje ponowny start odbioru w trybie przerwania, co umożliwia ciągłą obsługę komunikacji bez blokowania wątków.

2.4.4 Reszta funkcjonalności

Resztę funkcjonalności można podejrzeć w dokumentacji Doxygen programu, w której szczegółowo opisano wszystkie funkcje. Dodano również bibliotekę CMSIS-DSP³, wykorzystywana do przetwarzania sygnałów i obliczeń matematycznych w zadaniach sterowania temperaturą i analizie danych z czujników.

3 Wnioski

3.1 Zrealizowane założenia projektowe

1. Podział kodu źródłowego na moduły oraz dokumentacja kodu zgodna ze standardem Doxygen, zapewniająca separację funkcjonalności i konfiguracji.
2. Wykorzystanie systemu kontroli wersji do zarządzania kodem źródłowym.
3. System zapewnia uchyb utrzymany na poziomie 1% zakresu regulacji.
4. Dedykowana aplikacja desktopowa pełniąca funkcję graficznego interfejsu użytkownika.
5. Dedykowane skrypty i modele do logowania sygnałów sterujących i pomiarowych.
6. Dodatkowe urządzenie wyjścia użytkownika – wyświetlacz TFT ST7735S.
7. Dodatkowe urządzenie wejścia użytkownika – potencjometr umożliwiający ręczne ustawienie wartości zadanej temperatury.
8. Protokół komunikacji szeregowej z wykorzystaniem sumy kontrolnej CRC.
9. Wykorzystanie systemu czasu rzeczywistego FreeRTOS do obsługi zadań pomiaru, sterowania i wyświetlania danych.

3.2 Wnioski końcowe

W ramach realizacji projektu opracowano system pomiaru i regulacji temperatury z wizualizacją graficzną oraz możliwością sterowania z poziomu aplikacji desktopowej. Projekt umożliwił praktyczne wykorzystanie mikrokontrolera STM32, wyświetlacza TFT ST7735S, czujników temperatury LM35DZ oraz elementów wykonawczych: grzałki i wentylatora.

³Ćwiczenia laboratoryjne nr 10, kurs AiR 2025/2026, portal eKursy.pl: <https://ekursy.put.poznan.pl/>

System osiąga założony uchyb regulacji na poziomie 1% zakresu pracy, co potwierdza prawidłowe działanie regulatora PID w warunkach laboratoryjnych.

Projekt pozwolił również na pogłębienie wiedzy z zakresu programowania mikrokontrolerów, obsługi interfejsów SPI i UART, oraz wykorzystania wyświetlaczy graficznych w systemach embedded.

4 Bibliografia

Poniższe źródła wykorzystano do opracowania projektu, dokumentacji kodu oraz materiałów dydaktycznych:

Dokumentacja i przykłady dotyczące przetworników ADC i wyświetlacza ST7735S oparte są na materiałach FORBOT [1, 2]. Dodatkowe ćwiczenia laboratoryjne z CMSIS-DSP wykorzystano zgodnie z materiałami udostępnionymi przez portal eKursy [3].

Literatura

- [1] FORBOT, “Kurs stm32l4 – przetworniki analogowo-cyfrowe adc,” <https://forbot.pl/blog/kurs-stm32l4-przetworniki-analogowo-cyfrowe-adc-id46587>, dostęp: 2026.
- [2] —, “Kurs stm32l4 – kolorowy wyświetlacz tft spi (st7735),” <https://forbot.pl/blog/kurs-stm32l4-kolorowy-wyswietlacz-tft-spi-id48620>, dostęp: 2026.
- [3] Politechnika Poznańska, “Ćwiczenia laboratoryjne nr 10 – cmsis-dsp,” <https://ekursy.put.poznan.pl/>, kierunek Automatyka i Robotyka, rok akad. 2025/2026.